

MANUAL TÉCNICO Y AUTOMATIZACIÓN

Estructura del Catálogo Digital Escalable & Script de Migración de Datos

1. Script de Automatización: Excel a JavaScript (Python)

Este script procesa un archivo de Excel (`.xlsx`), extrae de forma automática los datos de las columnas y genera un archivo formateado en JavaScript listo para ser inyectado en el archivo de lógica del catálogo, clasificando las marcas y líneas mediante patrones de texto.

```

import pandas as pd
import json

archivo_excel = 'catalogo.xlsx'

try:
    df = pd.read_excel(archivo_excel)
    productos_js = []

    for index, row in df.iterrows():
        codigo = str(row.iloc[0]).strip().split('.')[0]
        try:
            precio_detal = float(row.iloc[1])
        except:
            precio_detal = 0.0

        try:
            precio_mayor = float(row.iloc[2])
        except:
            precio_mayor = 0.0

        descripcion = str(row.iloc[3]).strip()

        if descripcion == "nan" or codigo == "nan":
            continue

        # Identificación automática de Marcas
        desc_lower = descripcion.lower()
        if 'ushas' in desc_lower:
            marca = 'Ushas'
        elif 'kevin' in desc_lower or 'coco' in desc_lower:
            marca = 'Kevin Coco'
        elif 'hudavioji' in desc_lower or 'huda' in desc_lower:
            marca = 'Hudavioji'
        else:
            marca = 'Otra'

        # Clasificación automática de Líneas/Categorías
        if 'rubor' in desc_lower or 'blush' in desc_lower:
            linea = 'rubores'
        elif 'sombra' in desc_lower or 'paleta' in desc_lower:
            linea = 'paletas'
        elif 'labial' in desc_lower or 'lip' in desc_lower:
            linea = 'labiales'
        else:
            linea = 'general'

        item = {
            "id": index + 1,

```

```
        "codigo": codigo,
        "precio_detal": round(precio_detal, 2),
        "precio_mayor": round(precio_mayor, 2),
        "descripcion": descripcion,
        "marca": marca,
        "linea": linea
    }
    productos_js.append(item)

with open('resultado_productos.js', 'w', encoding='utf-8') as f:
    f.write("const productos = ")
    json.dump(productos_js, f, ensure_ascii=False, indent=2)
    f.write(";")

print(f"Éxito: Procesados {len(productos_js)} productos.")

except Exception as e:
    print(f"Error al procesar el archivo: {e}")
```

2. Estructura del Proyecto Arquitectura Decoupled

Para garantizar que el sistema soporte un alto volumen de productos sin degradar el rendimiento del navegador, los entornos de desarrollo se separan estrictamente de la siguiente manera:

- **index.html** : Estructura semántica pura del DOM. No contiene elementos repetitivos de productos duros en el código.
- **styles.css** : Estilos globales, diseño Grid/Flexbox responsivo y directivas de renderizado visual de tarjetas.
- **app.js** : Lógica de control de estados, base de datos en formato de arreglo de objetos y motores de render dinámico.

3. Mecanismo Lógico de Filtrado Unificado

El núcleo del filtrado interactivo opera interceptando los eventos de cambio en la barra de búsqueda y selectores. Realiza una operación lógica **AND** entre todas las condiciones en una sola iteración:

```
function filtrarProductos() {
  const query = document.getElementById("searchInput").value.toLowerCase();
  const marca = document.getElementById("marcaSelect").value;
  const categoria = document.getElementById("categoriaSelect").value;

  const filtrados = productos.filter(p => {
    const matchText = p.codigo.toLowerCase().includes(query) ||
                      p.descripcion.toLowerCase().includes(query);
    const matchMarca = (marca === "todas") || (p.marca === marca);
    const matchCategoria = (categoria === "todas") || (p.linea === categoria);

    return matchText && matchMarca && matchCategoria;
  });
  renderizarProductos(filtrados);
}
```

4. Protocolo de Crecimiento y Escalabilidad

Escenario A: Inserción de productos en categorías existentes

Cuando entren nuevos productos pertenecientes a marcas o líneas ya configuradas, el proceso se limita estrictamente a la capa de datos. Basta con añadir el nuevo elemento al arreglo de datos en **app.js** (o regenerarlo mediante el script de Python). El motor de filtrado lo asimilará dinámicamente sin necesidad de modificar el HTML.

Escenario B: Incorporación de una nueva línea comercial (Ej: Labiales)

Si la empresa expande su catálogo a un segmento de productos completamente nuevo, se debe ejecutar un protocolo de sincronización de dos pasos:

Paso 1: Modificar el archivo `index.html` añadiendo la opción correspondiente al elemento de selección de categorías, asignando en el atributo `value` el identificador técnico exacto:

```
<select id="categoriaSelect" onchange="filtrarProductos()">
  <option value="todas">Todas las categorías</option>
  <option value="labiales">Labiales 📌</option>
</select>
```

Paso 2: Al estructurar los nuevos artículos dentro de la base de datos, mapear la propiedad `linea` con el string exacto utilizado en el paso anterior:

```
{
  "id": 102,
  "codigo": "6924372611111",
  "precio_detal": 3.50,
  "precio_mayor": 2.80,
  "descripcion": "Labial Premium Glitter Ushas",
  "marca": "Ushas",
  "linea": "labiales"
}
```

5. Mantenimiento y Rendimiento Técnico

- **Optimización de Carga:** Las imágenes vinculadas mediante el identificador dinámico deben ser procesadas en formato `.webp` a una resolución sugerida de 400x400 px para mitigar el consumo de datos móviles.
- **Persistencia de Datos:** El recálculo de divisas opera sobre variables en memoria sincronizadas con el `localStorage`, permitiendo mantener la integridad de los montos y el estado del carrito de compras del usuario ante actualizaciones accidentales.